

RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution, Affiliated to Anna
University Rajalakshmi Nagar, Thandalam – 602-
105



DEPARTMENT OF INFORMATION TECHNOLOGY

IT23431 - MONGODB ESSENTIALS

(Regulation 2023)

LAB MANUAL

Name :

Register No. :

Year / Branch / Section :

Semester :

Academic Year :

Course Code	Course Title (Laboratory Integrated Theory Course)	L	T	P	C
IT23431	MongoDB Essentials	2	0	2	3

LIST OF EXPERIMENTS			
SNO	NAME OF THE EXPERIMENT	PAGE NO	SIGN
1	MongoDB installation and basic setup		
2	Creating and dropping tables with various constraints		
3	Performing CRUD Operations A] StudentMarkDetails B] EmployeeSalaryDetails C] Sales Details		
4	A] Projection B] Aggregate		
5	Data Modeling		
6	Perform Schema Design and Indexing		
7	Building aggregation pipelines		
8	Performing Data Transformation		
9	Establishing connection between MongoDB and java		
10	Develop a CRUD Application using MongoDB(Mini Project)		

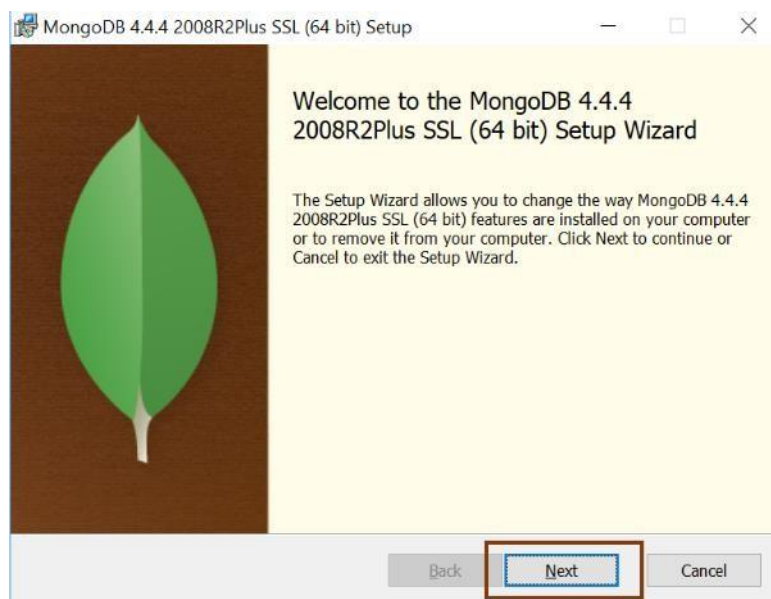
Ex. Nos. 1	MONGO DB INSTALLATION AND BASIC SET UP
Date :	

Aim:

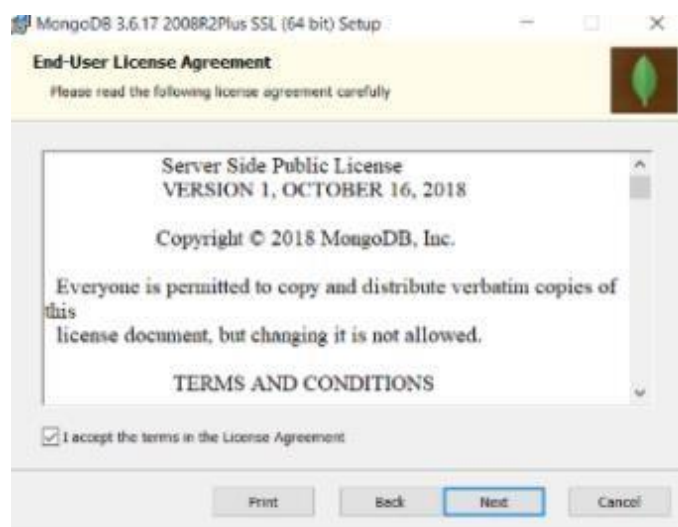
To study the installation of MongoDB setup environment.

Procedure for installation:

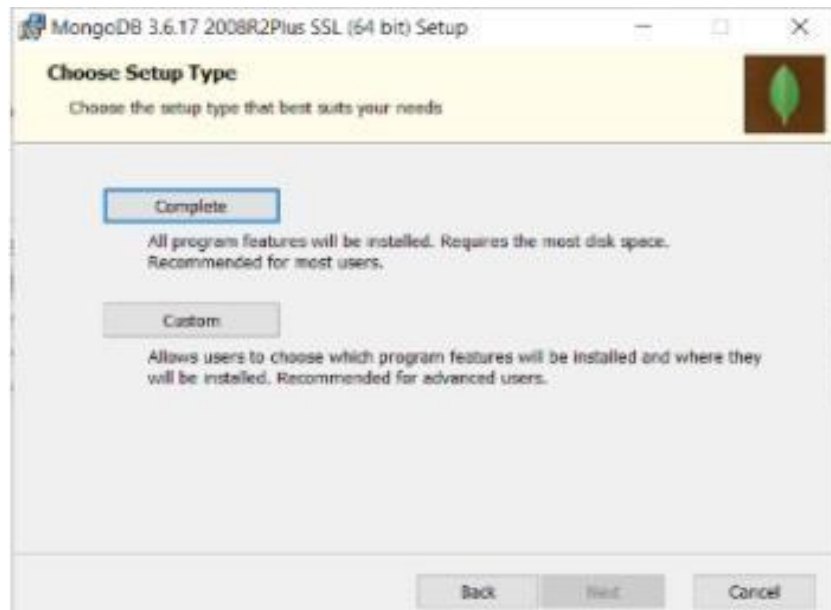
1. Download the installer.
2. Link : <https://www.mongodb.com/docs/manual/installation/>
3. Run the installer.



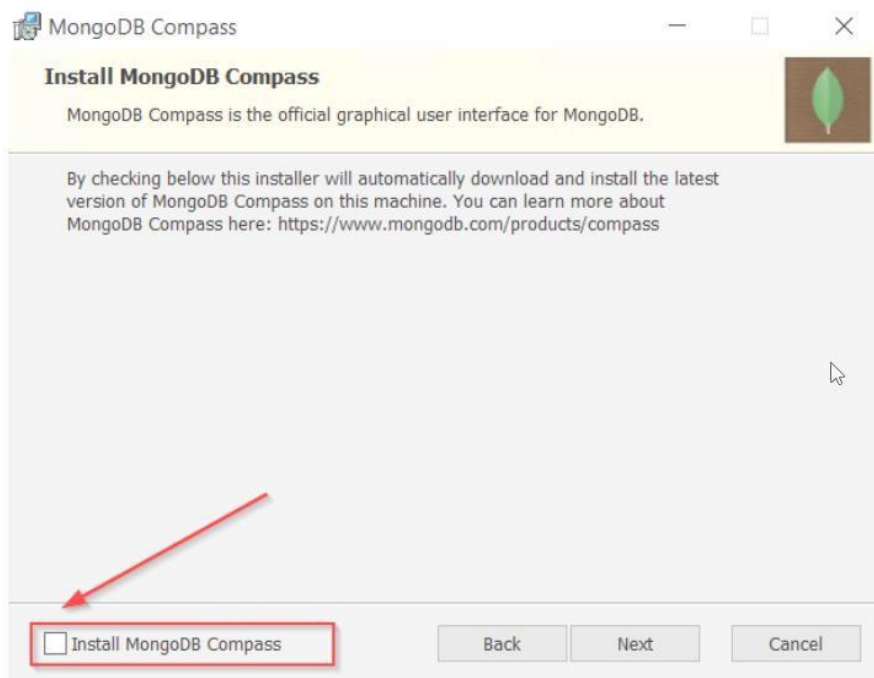
4. Accept the terms in the License agreement and click out.



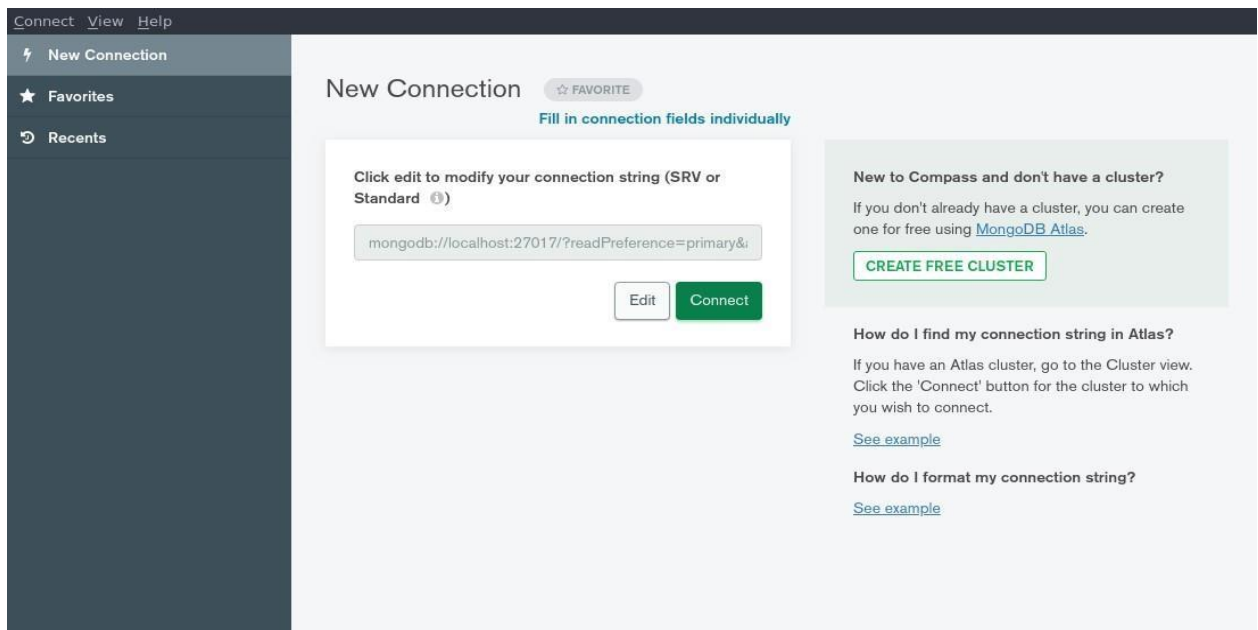
5. Choose the setup type as Complete and click next.



6. Install MongoDB Compass.



7. Click next twice.
8. Click Install and finish.



9. Connect the database.
10. The installation is complete and basic set up is done.

Result:

Thus the MongoDB installation and basic setup environment is successfully complete.

Ex. No. 2	CREATING AND DROPPING TABLES WITH VARIOUS CONSTRAINTS
Date:	

Aim:

To create and drop collection with various constraints.

Procedure:

CREATE

1. Creating a database using the Use <db> command: syntax :

use <databaseName>

Query: use employee

Output:

>switch to employee

If myDatabase does not exist, MongoDB will create it when data is inserted.

2. Creating a collection using the db.createCollection("<collectionName>") command :

The db.createCollection("<collectionName>") command in MongoDB creates a new collection with the specified name in the current database.

syntax : db.createCollection("<collectionName>") Query:

db.createCollection("employees")

Output:

>{ ok :1}

.DROP

Dropping a collection using db.<collectionName>.drop() the command:

The db.<collectionName>.drop() command in MongoDB permanently

deletes the specified collection and all its documents from the database

syntax : **db.<collectionName>.drop()**

Query: **db.employees.drop()** **Output**

```
>db.employees.drop
```

```
< true
```

Dropping a database using the db.dropDatabase() command:

The db.dropDatabase() command is used to permanently delete the selected database along with all its collections and documents.

Before executing, ensure you have switched to the correct database using the use database_name command.

syntax : **db.dropDatabase()** **Query:**

db.dropDatabase() **Output:**

```
>use employee
```

```
< switched to db employee
```

```
>db.dropDatabase()
```

```
< { ok : 1, dropped : 'employee' }
```

Result:

Thus the creating and dropping database and collection on MongoDB compass was execute successfully.

Ex. No. 3	CRUD OPERATIONS
Date:	

Aim:

To program CRUD operation on three collection in MongoDB Compass.

PROCEDURE:

Create Collection “students”:

```
>db.createCollection(“students”)
```

Create:

InsertOne:

Query:

```
db.students.insertOne({ name: "Alice", rollNo: 1, course: "CS", marks: 85 });
```

```
{
```

```
  "acknowledged": true,
```

```
  "insertedId": ObjectId("67f6a55013545e3b0785613a")
```

```
}
```

InsertMany:

Query:

```
db.students.insertMany([
```

```
  { name: "Bob", rollNo: 2, course: "Math", marks: 78 },
```

```
  { name: "Carol", rollNo: 3, course: "Physics", marks: 88 }
]);
```

```
{
```

```
{
```

```
  "acknowledged": true,
```

```
  "insertedIds": {
```

```
    "0": ObjectId("67f6a57413545e3b0785613b"),
```

```
    "1": ObjectId("67f6a57413545e3b0785613c")
```

```
  }}
```

```
>db.students.find();

[
  {
    "_id": ObjectId("67f6a55013545e3b0785613a"),
    "name": "Alice",
    "rollNo": 1,
    "course": "CS",
    "marks": 85
  },
  {
    "_id": ObjectId("67f6a57413545e3b0785613b"),
    "name": "Bob",
    "rollNo": 2,
    "course": "Math",
    "marks": 78
  },
  {
    "_id": ObjectId("67f6a57413545e3b0785613c"),
    "name": "Carol",
    "rollNo": 3,
    "course": "Physics",
    "marks": 88
  }
]
```

Update

UpdateOne

```
db.students.updateOne({ name: "Alice" }, { $set: { marks: 90 } });

{
  "acknowledged": true,
  "insertedId": null,
  "matchedCount": 1,
```

```
"modifiedCount": 1,  
"upsertedCount": 0  
}
```

UpdateMany

Query:

```
db.students.updateMany({}, { $inc: { marks: 5 } });
```

Output:

```
>db.students.find();
```

Delete DeleteOne

```
db.students.deleteOne({ rollNo: 2 });
```

```
{  
  "acknowledged": true,  
  "deletedCount": 1  
}  
  
{  
  "_id": ObjectId("6776a55013545e3b0785613a"),  
  "name": "Alice",  
  "rollNo": 1,  
  "course": "CS",  
  "marks": 95  
}  
  
{  
  "_id": ObjectId("6776a57413545e3b0785613c"),  
  "name": "Carol",  
  "rollNo": 3,  
  "course": "Physics",  
  "marks": 93  
}
```

DeleteMany:

Query

```
db.students.deleteMany({ course: "Physics" });
```

```
{
  acknowledged: true,
  deletedCount: 1
}
```

Output:

Query:

```
{
  "_id": ObjectId("676a555013545e3b0785613a"),
  "name": "Alice",
  "rollNo": 1,
  "course": "CS",
  "marks": 95
}
```

Create collection “employee”

```
>db.createCollection(“employees”);
```

Create:

InsertOne:

```
db.employees.insertOne({ name: "John", position: "Developer", salary: 40000 });
```

```
{
  "acknowledged": true,
  "insertedId": ObjectId("67f6ac3abd60c89ad017ca54")
}
```

InsertMany

```
db.employees.insertMany([
  { name: "Emma", position: "Manager", salary: 60000 },
  { name: "Liam", position: "Analyst", salary: 45000 }
]);
{
  "acknowledged": true,
```

```
"insertedIds": {  
  "0": ObjectId("G7f6ac76bd60c89ad017ca55"),  
  "1": ObjectId("G7f6ac76bd60c89ad017ca56")  
}}
```

```
{
  "_id": ObjectId("676ac3abd60c89ad017ca54"),
  "name": "John",
  "position": "Developer",
  "salary": 40000
}

{
  "_id": ObjectId("676ac76bd60c89ad017ca55"),
  "name": "Emma",
  "position": "Manager",
  "salary": 60000
}

{
  "_id": ObjectId("676ac76bd60c89ad017ca56"),
  "name": "Liam",
  "position": "Analyst",
  "salary": 45000
}
```

UpdateOne

```
db.employees.updateOne({ name: "John" }, { $set: { salary: 42000 } });
```

```
{
  "acknowledged": true,
  "insertedId": null,
  "matchedCount": 1,
  "modifiedCount": 1,
  "upsertedCount": 0}
```

UpdateMany

```
db.employees.updateMany({}, { $inc: { salary: 1000 } });
```

```
{  
  "acknowledged": true,  
  "insertedId": null,  
  "matchedCount": 3,  
  "modifiedCount": 3,  
  "upsertedCount": 0  
}
```

Output:

```
{  
  "_id": ObjectId("67f6ac3abd60c89ad017ca54"),  
  "name": "John",  
  "position": "Developer",  
  "salary": 43000  
}  
  
{  
  "_id": ObjectId("67f6ac7dbd60c89ad017ca55"),  
  "name": "Emma",  
  "position": "Manager",  
  "salary": 61000  
}  
  
{  
  "_id": ObjectId("67f6ac7dbd60c89ad017ca56"),  
  "name": "Liam",  
  "position": "Analyst",  
  "salary": 46000  
}
```

DeleteOne

```
db.employees.deleteOne({ name: "Liam" });

{
  "acknowledged": true,
  "deletedCount": 1
}
```

DeleteMany

```
db.employees.deleteMany({ position: "Manager" });

{
  "acknowledged": true,
  "deletedCount": 1
}
```

Output

```
{
  "_id": ObjectId("67f6ac3abd60c89ad017ca54"),
  "name": "John",
  "position": "Developer",
  "salary": 43000
}
```

Create a collection “Sales”

```
>db.create.Collection(“Sales”);
```

Create

```
InsertOne;
db.sales.insertOne({ product: "Mobile", quantity: 5, price: 15000 });

{
```

```
"acknowledged": true,  
"insertedId": ObjectId("67f6af46bd60c89ad017ca57")  
}
```

InsertMany:

```
db.sales.insertMany([  
  { product: "Laptop", quantity: 2, price: 80000 },  
  { product: "Tablet", quantity: 3, price: 25000 }  
]);
```

```
{  
  "acknowledged": true,  
  "insertedIds": {  
    "0": ObjectId("G7f6af64bd60c89ad017ca58"),  
    "1": ObjectId("G7f6af64bd60c89ad017ca59")  
  }  
}
```

Output

```
> db.sales.find()  
  
{  
  _id: ObjectId('67f6af46bd60c89ad017ca57'),  
  product: 'Mobile',  
  quantity: 5,  
  price: 15000  
},  
_id: ObjectId('67f6af64bd60c89ad017ca58'), product: 'Laptop',  
  quantity: 2,  
  price: 80000  
}
```

```
{
  _id: ObjectId('67f6af64bd60c89ad017ca59'),
  product: 'Tablet',

  quantity: 3,
  price: 25000
}
```

Update:

UpdateOne:

```
db.sales.updateOne({ product: "Mobile" }, { $set: { quantity: 6 } });
> db.sales.updateOne({
  product: "Mobile" }, { $set: { quantity: 6 } });
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

UpdateMany:

```
db.sales.updateMany({}, { $inc: { price: 1000 } });
db.sales.updateMany([], { $inc: { price: 1000 } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
}
```

```
modifiedCount: 3,
```

```
upsettedCount: 0
```

```
}
```

Output:

```
db.sales.find()
```

```
{
```

```
  -id: ObjectId('67f6af46bd60c89ad017ca57'),
```

```
  product: 'Mobile',
```

```
  quantity: 6,
```

```
  price: 16000
```

```
}
```

```
{
```

```
  -id: ObjectId('67f6af64bd60c89ad017ca58'),
```

```
  product: 'Laptop',
```

```
  quantity: 2,
```

```
  price: 81000
```

```
}
```

```
  -id: ObjectId('67f6af64bd60c89ad017ca59'),
```

```
  product: 'Tablet',
```

```
  quantity: 3,
```

```
  price: 26000
```

```
}
```

Delete

DeleteOne:

```
db.sales.deleteOne({ product: "Tablet" });
```

```
{
```

```
"acknowledged": true,  
"deletedCount": 1  
}
```

DeleteMany:

```
db.sales.deleteMany({ price: { $gt: 70000 } });  
  
{  
  "acknowledged": true,  
  "deletedCount": 1  
}
```

Output

```
db.sales.find()  
  
{  
  -id: ObjectId('676af46bd60c89ad017ca57'),  
  product: 'Mobile',  
  quantity: 6,  
  price: 16000  
}
```

Result:

Thus, the above queries to perform CRUD operations on various collections is executed successfully.

Ex. No. 4a	PROJECTION
Date:	

Aim:

To perform an projection operation on Employee collection.

PROCEDURE:

Create a collection “Employee”

```
>db.createCollection("Employee")
```

Insert the data

```
db.Employee.insertMany([
  {
    name: "Alice",
    age: 30,
    department: "HR",
    skills: ["Communication", "Recruitment", "Conflict Resolution"],
    projects: [
      { title: "Onboarding", status: "Completed" },
      { title: "Payroll Setup", status: "In Progress" }
    ]
  },
  {
    name: "Bob",
    age: 25,
    department: "IT",
    skills: ["JavaScript", "Node.js", "MongoDB"],
    projects: [
      { title: "Website Revamp", status: "Completed" },
      { title: "Security Audit", status: "Pending" }
    ]
  }
])
```

```
    },  
  
    {  
      name: "Charlie",  
      age: 35,  
      department: "Finance",  
      skills: ["Accounting", "Excel", "Analysis"],  
      projects: [  
        { title: "Budget Planning", status: "Completed" },  
        { title: "Tax Filing", status: "Completed" }  
      ]  
    }  
  ]  
})
```

\$Match

\$match filters documents in the pipeline based on given conditions. It works like the find() method and is used to narrow down data early in the aggregation process.

Query:

```
db.Employee.aggregate([  
  {  
    "$match": {  
      "department": "IT"  
    }  
  }  
])
```

Output

```
[
  {
    "_id": 1,
    "name": "Alice",
    "department": "IT",
    "skills": ["MongoDB", "Node.js", "JavaScript"],
    "projects": [...]
  },
  {
    "_id": 3,
    "name": "Charlie",
    "department": "IT",
    "skills": ["MongoDB", "Express", "React", "Node.js"],
    "projects": [...]
  }
]
```

\$ElemMatch

\$elemMatch is used to match **specific elements inside an array field** that meet certain criteria. It ensures that **all conditions** apply to the **same array element**.

```
db.Employee.aggregate([
  {
    "$match": {
      "projects": {
        "$elemMatch": {
          "status": "Pending"
        }
      }
    }
  })
```

Output

```
[
  {
    "_id": 1,
    "name": "Alice",
    "projects": [
      { "name": "Project A", "status": "Completed" },
      { "name": "Project B", "status": "Pending" }
    ]
  },
  {
    "_id": 3,
    "name": "Charlie",
    "projects": [
      { "name": "Project C", "status": "Pending" }
    ]
  }
]
```

\$Slice

\$slice returns a **subset of array elements**. You can specify how many items to return from the start or use it like pagination (with skip + limit).

Query

```
db.Employee.aggregate([
  {
    "$project": {
      "name": 1,
      "skills": { "$slice": ["$skills", 2] }
    }
  })
```

Output

```
[
  { "name": "Alice", "skills": ["MongoDB", "Node.js"] },
  { "name": "Bob", "skills": ["Communication", "Recruitment"] },
  { "name": "Charlie", "skills": ["MongoDB", "Express"] }
]
```

\$Meta

\$meta is used to **return metadata** about documents — typically used with **text search** to get the **relevance score** (textScore) of how well a document matches a search term.

Make sure the Text Index exists

```
db.Employee.createIndex({ skills: "text" })
```

Query

```
db.Employee.aggregate([
  {
    "$match": {
      "$text": { "$search": "MongoDB" }
    }
  },
  {
    "$project": {
      "name": 1,
      "skills": 1,
      "score": { "$meta": "textScore" }
    }
  },
  {
    "$sort": {
      "score": { "$meta": "textScore" } } })
```

Output:

```
[
  {
    "name": "Alice",
    "skills": ["MongoDB", "Node.js", "JavaScript"],
    "score": 1.5
  },
  {
    "name": "Charlie",
    "skills": ["MongoDB", "Express", "React", "Node.js"],
    "score": 1.5
  }
]
```

Result:

Thus, the queries to perform various Projection operations on Employee collection is executed successfully.

Ex. No. 4b	AGGREGATION
Date:	

Aim:

To perform an aggregate operation on Employee collection.

PROCEDURE:

Insert

```
db.Sales.insertMany([
  { item: "Laptop", category: "Electronics", price: 1000, quantity: 2, store: "Store A" },
  { item: "Phone", category: "Electronics", price: 500, quantity: 5, store: "Store B" },
  { item: "Shoes", category: "Fashion", price: 100, quantity: 10, store: "Store A" },
  { item: "T-shirt", category: "Fashion", price: 20, quantity: 20, store: "Store B" },
  { item: "TV", category: "Electronics", price: 1500, quantity: 1, store: "Store C" },
  { item: "Smartwatch", category: "Electronics", price: 300, quantity: 3, store: "Store A" }
])
```

1. \$match

```
db.Sales.aggregate([
  { $match: { category: "Electronics" } }
])
```

Output

```
[
  { item: "Laptop", category: "Electronics", price: 1000, quantity: 2, store: "Store A" },
  { item: "Phone", category: "Electronics", price: 500, quantity: 5, store: "Store B" },
  { item: "TV", category: "Electronics", price: 1500, quantity: 1, store: "Store C" }
]
```

2. \$project

```
db.Sales.aggregate([
  {
    $project: {
      item: 1,
      price: 1,
      quantity: 1,
```

```

        totalSale: { $multiply: ["$price", "$quantity"] }
      }
    }
  })
)

```

Output

```

[
  { item: "Laptop", price: 1000, quantity: 2, totalSale: 2000 },
  { item: "Phone", price: 500, quantity: 5, totalSale: 2500 },
]

```

3. \$group

```

db.Sales.aggregate([
  {
    $group: {
      _id: "$store",
      totalQuantity: { $sum: "$quantity" },
      averagePrice: { $avg: "$price" }
    }
  }
])

```

Output

```

[
  { _id: "Store A", totalQuantity: 15, averagePrice: 550 },
  { _id: "Store B", totalQuantity: 25, averagePrice: 260 },
  { _id: "Store C", totalQuantity: 1, averagePrice: 1500 }
]

```

4. \$sum

```

db.Sales.aggregate([
  {
    $group: {
      _id: "$category",
      totalQuantity: { $sum: "$quantity" }
    }
  }
])
)

```

Output

```

[
  { _id: "Electronics", totalQuantity: 11 },
  { _id: "Fashion", totalQuantity: 30 }
]

```

5. \$avg

```
db.Sales.aggregate([
  {
    $group: {
      _id: "$category",
      avgPrice: { $avg: "$price" }
    }
  }
])
```

Output

```
[
  { _id: "Electronics", avgPrice: 825 },
  { _id: "Fashion", avgPrice: 60 }
]
```

6. \$min

```
db.Sales.aggregate([
  {
    $group: {
      _id: "$category",
      minPrice: { $min: "$price" }
    }
  }
])
```

Output

```
[
  { _id: "Electronics", minPrice: 300 },
  { _id: "Fashion", minPrice: 20 }
]
```

7. \$max

```
db.Sales.aggregate([
  {
    $group: {
      _id: "$category",
```

```
      maxPrice: { $max: "$price" }
    }
  }
})
```

Output

```
[
  { _id: "Electronics", maxPrice: 1500 },
  { _id: "Fashion", maxPrice: 100 }
]
```

8. \$sort

```
db.Sales.aggregate([
  { $sort: { price: -1 } }
])
```

Output

```
[
  { item: "TV", price: 1500, ... },
  { item: "Laptop", price: 1000, ... },
  { item: "Phone", price: 500, ... },
]
```

Result:

Thus, the query to perform various Aggregation operations on Sales collection are executed successfully.

Ex. No. 5	DATA MODELING
Date:	

Aim:

To implement embedded and reference model example using inventory collection.

Embedded Model

Procedure:

```
db.Inventory.insertMany([
  {
    _id: 1,
    item: "Laptop",
    category: "Electronics",
    currentStock: 10,
    stockHistory: [
      { date: "2024-03-01", change: 5, reason: "Restock" },
      { date: "2024-03-05", change: -2, reason: "Sold" }
    ]
  },
  {
    _id: 2,
    item: "Phone",
    category: "Electronics",
    currentStock: 20,
    stockHistory: [
      { date: "2024-03-02", change: 10, reason: "Restock" },
      { date: "2024-03-07", change: -3, reason: "Sold" },
      { date: "2024-03-10", change: -2, reason: "Damaged" }
    ]
  },
  {
    _id: 3,
```

```
    item: "Shoes",
    category: "Fashion",
    currentStock: 15,
    stockHistory: [
      { date: "2024-03-01", change: 15, reason: "Restock" },
      { date: "2024-03-06", change: -5, reason: "Sold" }
    ]
  }
])
```

Get latest stock change for each item:

```
db.Inventory.aggregate([
  {
    $project: {
      item: 1,
      currentStock: 1,
      lastChange: { $arrayElemAt: ["$stockHistory", -1] }
    }
  }
])
```

Output

```
[
  {
    item: "Laptop",
    currentStock: 10,
    lastChange: { date: "2024-03-05", change: -2, reason: "Sold" }
  },
  {
    item: "Phone",
    currentStock: 20,
    lastChange: { date: "2024-03-10", change: -2, reason: "Damaged" }
  }
]
```

```
    },  
    {  
      item: "Shoes",  
      currentStock: 15,  
      lastChange: { date: "2024-03-06", change: -5, reason: "Sold" }  
    }  
  ]  
}
```

Count number of stock changes per item:

```
db.Inventory.aggregate([  
  {  
    $project: {  
      item: 1,  
      changesCount: { $size: "$stockHistory" }  
    }  
  }  
])
```

Output:

```
[  
  { item: "Laptop", changesCount: 2 },  
  { item: "Phone", changesCount: 3 },  
  { item: "Shoes", changesCount: 2 }  
]
```

Reference Data Model

Procedure:

```
db.Inventory.insertMany([  
  { _id: 1, item: "Laptop", category: "Electronics", currentStock: 10 },  
  { _id: 2, item: "Phone", category: "Electronics", currentStock: 20 },  
  { _id: 3, item: "Shoes", category: "Fashion", currentStock: 15 }  
])
```

```
db.StockHistory.insertMany([
  { _id: 101, itemId: 1, date: "2024-03-01", change: 5, reason: "Restock" },
  { _id: 102, itemId: 1, date: "2024-03-05", change: -2, reason: "Sold" },

  { _id: 103, itemId: 2, date: "2024-03-02", change: 10, reason: "Restock" },
  { _id: 104, itemId: 2, date: "2024-03-07", change: -3, reason: "Sold" },
  { _id: 105, itemId: 2, date: "2024-03-10", change: -2, reason: "Damaged" },

  { _id: 106, itemId: 3, date: "2024-03-01", change: 15, reason: "Restock" },
  { _id: 107, itemId: 3, date: "2024-03-06", change: -5, reason: "Sold" }
])
```

Join Inventory with StockHistory:

```
db.Inventory.aggregate([
  {$lookup: {
    from: "StockHistory",
    localField: "_id",
    foreignField: "itemId",
    as: "stockHistory"
  } })
```

Output:

```
{
  _id: 1,
  item: "Laptop",
  currentStock: 10,
  stockHistory: [
    { itemId: 1, date: "2024-03-01", change: 5, reason: "Restock" },
    { itemId: 1, date: "2024-03-05", change: -2, reason: "Sold" }
  ]
}
```

Stock changes in March 2024:

```
db.StockHistory.find({  
  date: { $gte: "2024-03-01", $lte: "2024-03-31" }  
})
```

Output:

```
[  
  { itemId: 1, date: "2024-03-01", change: 5, reason: "Restock" },  
  { itemId: 1, date: "2024-03-05", change: -2, reason: "Sold" },  
  ...  
]
```

Total stock added per item:

```
db.StockHistory.aggregate([  
  { $match: { change: { $gt: 0 } } },  
  {  
    $group: {  
      _id: "$itemId",  
      totalAdded: { $sum: "$change" }}}])
```

Output:

```
[  
  { _id: 1, totalAdded: 5 },  
  { _id: 2, totalAdded: 10 },  
  { _id: 3, totalAdded: 15 }  
]
```

List items with stock reduction and their names:

```
db.StockHistory.aggregate([
  { $match: { change: { $lt: 0 } } },
  {
    $lookup: {
      from: "Inventory",
      localField: "itemId",
      foreignField: "_id",
      as: "itemInfo"
    }
  },
  { $unwind: "$itemInfo" },
  {
    $project: {
      item: "$itemInfo.item",
      change: 1,
      reason: 1,
      date: 1
    }
  }
])
```

Output:

```
[
  { item: "Laptop", change: -2, reason: "Sold", date: "2024-03-05" },
  { item: "Phone", change: -3, reason: "Sold", date: "2024-03-07" },
  { item: "Phone", change: -2, reason: "Damaged", date: "2024-03-10" },
  { item: "Shoes", change: -5, reason: "Sold", date: "2024-03-06" }
]
```

Result:

Thus, the queries to implement data modeling on inventory is executed successfully.

Ex. No. 6	PERFORM SCHEMA DESIGN AND INDEXING
Date:	

Aim

To perform Schema Design and Indexing in MongoDB.

Schema Design:

Let's assume an E-Commerce Product Database.
We'll create a "products" collection with fields like:

- `_id`
- `name`
- `category`
- `price`
- `stock`
- `ratings`

Data Insertion:

```
db.products.insertMany([  
  { _id: 1, name: "Laptop", category: "Electronics", price: 50000, stock: 25, ratings: 4.5 },  
  { _id: 2, name: "Mobile Phone", category: "Electronics", price: 20000, stock: 50, ratings: 4.2 },  
  { _id: 3, name: "T-shirt", category: "Fashion", price: 800, stock: 100, ratings: 4.0 },  
  { _id: 4, name: "Blender", category: "Home Appliances", price: 3000, stock: 15, ratings: 4.1 },  
  { _id: 5, name: "Bookshelf", category: "Furniture", price: 7000, stock: 10, ratings: 4.3 }  
]);
```

Indexing:

1. Create index on the category field to speed up category-based queries:

Query

```
>db.products.createIndex({ category: 1 });
```

2. Create compound index on price and ratings:

Query

```
>db.products.createIndex({ price: 1, ratings: -1 });
```

Queries:

1. Find all products in "Electronics" category:

```
>db.products.find({ category: "Electronics" });
```

output:

```
[
  { "_id": 1, "name": "Laptop", "category": "Electronics", "price": 50000, "stock": 25, "ratings": 4.5 },
  { "_id": 2, "name": "Mobile Phone", "category": "Electronics", "price": 20000, "stock": 50,
"ratings": 4.2 }
]
```

2. Find products with price less than 10000:

```
>db.products.find({ price: { $lt: 10000 } });
```

output:

```
[
  { "_id": 3, "name": "T-shirt", "category": "Fashion", "price": 800, "stock": 100, "ratings": 4.0 },
  { "_id": 4, "name": "Blender", "category": "Home Appliances", "price": 3000, "stock": 15, "ratings":
4.1 }
]
```

3. Check all created indexes:

```
>db.products.getIndexes();
```

output:

```
[
  { "v": 2, "key": { "_id": 1 }, "name": "_id_ " },
  { "v": 2, "key": { "category": 1 }, "name": "category_1" },
  { "v": 2, "key": { "price": 1, "ratings": -1 }, "name": "price_1_ratings_-1" }
]
```

Result:

To perform Schema design and indexing in MongoDB were successfully performed.

Ex. No. 7	BUILDING AGGREGATION PIPELINES
Date:	

Aim:

To perform an aggregation pipelines operation for Students and Employees collections in MongoDB.

Student Collection:

```
db.students.insertMany([
  { studentId: 1, name: "Alice", age: 20, dept: "CSE", marks: 85 },
  { studentId: 2, name: "Bob", age: 22, dept: "ECE", marks: 78 },
  { studentId: 3, name: "Charlie", age: 21, dept: "CSE", marks: 92 },
  { studentId: 4, name: "David", age: 23, dept: "MECH", marks: 67 },
  { studentId: 5, name: "Eve", age: 20, dept: "ECE", marks: 88 }
])
```

Aggregation pipeline queries:

1. Filter students with marks greater than 80

```
>db.students.aggregate([
  { $match: { marks: { $gt: 80 } } }
])
```

Output:

```
{
  _id: ObjectId('680db7f5e45602d189c718ea'),
  studentId: 1,
  name: 'Alice',
  age: 20,
  dept: 'CSE',
  marks: 85
}
{
  _id: ObjectId('680db7f5e45602d189c718ec'),
  studentId: 3,
  name:
  'Charlie', age:
  21,
```

```
    dept: 'CSE',
    marks: 92
  }
  {
    _id: ObjectId('680db7f5e45602d189c718ee'),
    studentId: 5,
    name:
    'Eve', age:
    20, dept:
    'ECE',
    marks: 88
  }
}
```

2. Group students by department and get average marks

```
>db.students.aggregate([
  {
    $group: {
      _id: "$dept",
      avgMarks: { $avg:
        "$marks" }, count: { $sum:
        1 }
    }
  }
])
```

Output:

```
{
  _id: 'CSE',
  avgMarks:
  88.5,
  count: 2
}
{
  _id: 'MECH',
  avgMarks: 67,
  count: 1
}
{
  _id: 'ECE',
  avgMarks:
  83,
  count: 2
}
```

3. Project student name and a new field 'grade'

```
>db.students.aggregate([
  {
    $project: {
      name: 1,
      marks:
      1,
      grade: {
        $switch: {
          branches:
          [
            { case: { $gte: ["$marks", 90] }, then: "A+" },
            { case: { $gte: ["$marks", 75] }, then: "A" },
            { case: { $gte: ["$marks", 60] }, then: "B" }

          ],
          default: "C"
        }
      }
    }
  })
```

Output:

```
{
  _id: ObjectId('680db7f5e45602d189c718ea'),
  name: 'Alice',
  marks:
  85,
  grade:
  'A'
}
{
  _id: ObjectId('680db7f5e45602d189c718eb'),
  name: 'Bob',
  marks:
  78,
  grade:
  'A'
}
{
  _id: ObjectId('680db7f5e45602d189c718ec'),
  name: 'Charlie',
  marks: 92,
  grade: 'A+'
}
{
  _id: ObjectId('680db7f5e45602d189c718ed'),
  name: 'David',
  marks:
  67,
  grade: 'B'
}
```

```
{
  _id: ObjectId('680db7f5e45602d189c718ee'),
  name: 'Eve',
  marks:
    88,
  grade:
    'A'
}
```

4. Sort students by marks descending.

```
>db.students.aggregate([
  { $sort: { marks: -1 } }
])
```

Output:

```
{
  _id: ObjectId('680db7f5e45602d189c718ec'),
  studentId: 3,
  name:
    'Charlie', age:
      21,
  dept: 'CSE',
  marks: 92
}
{
  _id: ObjectId('680db7f5e45602d189c718ee'),
  studentId: 5,
  name:
    'Eve', age:
      20, dept:
        'ECE',
  marks: 88
}
{
  _id: ObjectId('680db7f5e45602d189c718ea'),
  studentId: 1,
  name: 'Alice',
  age: 20,
  dept: 'CSE',
  marks: 85
}
{
  _id: ObjectId('680db7f5e45602d189c718eb'),
  studentId: 2,
  name:
    'Bob', age:
      22, dept:
        'ECE',
  marks: 78
}
```

```
{
  _id: ObjectId('680db7f5e45602d189c718ed'),
  studentId: 4,
  name: 'David',
  age: 23,
  dept:
    'MECH',
  marks: 67
}
```

Employees Collection:

```
db.employees.insertMany([
  { empId: 101, name: "Mr. Smith", dept: "CSE", salary: 60000 },
  { empId: 102, name: "Mrs. Johnson", dept: "ECE", salary: 58000 },
  { empId: 103, name: "Mr. Lee", dept: "MECH", salary: 62000 },
  { empId: 104, name: "Ms. Brown", dept: "CSE", salary: 64000 }
])
```

1. Filter employees with salary above

```
60000. db.employees.aggregate([
  { $match: { salary: { $gt: 60000 } } }
])
```

Output:

```
{
  _id: ObjectId('680dbd10ee7dd40f070c6d22'),
  empId: 103,
  name: 'Mr. Lee',
  dept:
    'MECH',
  salary:
    62000
}
```

```
{
  _id: ObjectId('680dbd10ee7dd40f070c6d23'),
  empId: 104,
  name: 'Ms. Brown',
  dept: 'CSE',
  salary:
    64000
}
```

2. Group by department and count employees.

```
>db.employees.aggregate([
  {
    $group: {
      _id: "$dept",
      totalEmployees: { $sum:
        1 },
      avgSalary: { $avg: "$salary" }
    }
  }
])
```

Output:

```
{
  _id: 'ECE',
  totalEmployees: 1,
  avgSalary: 58000
}

{
  _id: 'MECH',
  totalEmployees: 1,
  avgSalary: 62000
}

{
  _id: 'CSE',
  totalEmployees: 2,
  avgSalary: 62000
}
```

3. Project name and salary with calculated tax

```
(10%) db.employees.aggregate([
  {
    $project: {
      name: 1,
      salary: 1,
      tax: { $multiply: ["$salary", 0.1] }
    }
  }
])
```

Output:

```
{
  _id: ObjectId('680dbd10ee7dd40f070c6d20'),
  name: 'Mr. Smith',
  salary: 60000,
  tax: 6000
}
{
  _id: ObjectId('680dbd10ee7dd40f070c6d21'),
  name: 'Mrs. Johnson',
  salary: 58000,
  tax: 5800
}
{
  _id: ObjectId('680dbd10ee7dd40f070c6d22'),
  name: 'Mr. Lee',
  salary: 62000,
  tax: 6200
}
{
  _id: ObjectId('680dbd10ee7dd40f070c6d23'),
  name: 'Ms. Brown',
  salary: 64000,
  tax: 6400
}
```

4. Sort by salary (high to low) and limit

```
top 3 db.employees.aggregate([
  { $sort: { salary: -1 } },
  { $limit: 3 }
])
```

Output

```
{
  _id: ObjectId('680dbd10ee7dd40f070c6d23'),
  empId: 104,
  name: 'Ms. Brown',
  dept: 'CSE',
  salary: 64000
}
```

```
{
  _id: ObjectId('680dbd10ee7dd40f070c6d22'),
  empId: 103,
  name: 'Mr. Lee',
  dept:
    'MECH',
  salary:
    62000
}
{
  _id: ObjectId('680dbd10ee7dd40f070c6d20'),
  empId: 101,
  name: 'Mr. Smith',
  dept: 'CSE',
  salary: 60000
}
```

Result:

Thus the above queries to perform an aggregation pipelines operation for Students and Employees collections in MongoDB is executed successfully.

Ex. No. 8	PERFORMING DATA TRANSFORMATION
Date:	

Aim:

To perform a Data transformation operation on “Stock” management system.

Stock collection:

```
db.stocks.insertMany([
  { item: "Laptop", category: "Electronics", quantity: 120, price: 55000 },
  { item: "Smartphone", category: "Electronics", quantity: 300, price: 25000 },
  { item: "Tablet", category: "Electronics", quantity: 150, price: 20000 },
  { item: "Office Chair", category: "Furniture", quantity: 80, price: 7000 },
  { item: "Desk", category: "Furniture", quantity: 50, price: 12000 },
  { item: "Water Bottle", category: "Utilities", quantity: 500, price: 200 },
  { item: "Notebook", category: "Stationery", quantity: 700, price: 40 },
  { item: "Pen", category: "Stationery", quantity: 1000, price: 15 },
  { item: "Backpack", category: "Bags", quantity: 90, price: 1500 },
  { item: "Printer", category: "Electronics", quantity: 60, price: 18000 }
])
```

1. Bucket (group items based on price

```
range): db.stocks.aggregate([
  {
    $bucket: {
      groupBy:
        "$price",
      boundaries: [0, 1000, 10000, 30000, 60000],
      default: "Other",
      output: {
        count: { $sum: 1 },
        items: { $push:
          "$item" }
      }
    }
  })
```

Output:

```
{
  _id: 0,
  count: 3,
  items: [
    'Water
    Bottle',
    'Notebook',
    'Pen'
  ]
}
{
  _id: 1000,
  count: 2,
  items: [
    'Office
    Chair',
    'Backpack'
  ]
}
{
  _id: 10000,
  count: 4,
  items: [
    'Smartphone'
    , 'Tablet',
    'Desk',
    'Printer'
  ]
}
{
  _id: 30000,
  count: 1,
  items: [
    'Laptop'
  ]
}
```

2. Facet (multiple aggregations in

```
one): db.stocks.aggregate([
  {
    $facet: {
      "ByCategory":
      [
        { $group: { _id: "$category", totalItems: { $sum: 1 } } }
      ],
      "HighQuantity": [
        { $match: { quantity: { $gte: 100 } } },
        { $project: { item: 1, quantity: 1, _id: 0 } }
      ]
    }
  })
```

Output:

```
{
  ByCategory: [
    {
      _id: 'Stationery',
      totalItems: 2
    },
    {
      _id: 'Furniture',
      totalItems: 2
    },
  ],
}
```

```
{
  _id: 'Bags',
  totalItems:
    1
},
{
  _id: 'Utilities',
  totalItems: 1
},
{
  _id: 'Electronics',
  totalItems: 4
}
],
HighQuantity: [
  {
    item: 'Laptop',
    quantity: 120,
  },
  {
    item: 'Smartphone',
    quantity: 300
  },
]
```

```
{
  item: 'Tablet',
  quantity: 150
},
{
  item: 'Water Bottle',
  quantity: 500
},
{
  item: 'Notebook',
  quantity: 700
},
{
  item: 'Pen',
  quantity: 1000
}
]
}
```

3. Sort by price (high to low):

```
db.stocks.find().sort({ price: -
1 })
```

Output:

```
{
  _id:
  ObjectId('680dc2ece6dd67696266cd1a'),
  item: 'Laptop',
  category: 'Electronics',
  quantity: 120,
  price: 55000
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd1b'),
  item: 'Smartphone',
  category: 'Electronics',
  quantity: 300,
  price: 25000
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd1c'),
  item: 'Tablet',
  category: 'Electronics',
  quantity: 150,
  price: 20000
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd23'),
  item: 'Printer',
  category: 'Electronics',
  quantity: 60,
  price: 18000
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd1e'),
  item: 'Desk',
  category: 'Furniture',
  quantity: 50,
  price: 12000
}
```

```
{
  _id:
  ObjectId('680dc2ece6dd67696266cd1d'),
  item: 'Office Chair',
  category: 'Furniture',
  quantity: 80,
  price: 7000
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd22'),
  item: 'Backpack',
  category: 'Bags',
  quantity: 90,
  price: 1500
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd1f'),
  item: 'Water Bottle',
  category: 'Utilities',
  quantity: 500,
  price: 200
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd20'),
  item: 'Notebook',
  category: 'Stationery',
  quantity: 700,
  price: 40
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd21'),
  item: 'Pen',
  category: 'Stationery',
  quantity: 1000,
  price: 15
}
```

4. Skip first 2 documents:

```
db.stocks.find().skip(
2)
```

Output:

```
{
  _id:
ObjectId('680dc2ece6dd67696266cd1c'),
  item: 'Tablet',
  category: 'Electronics',
  quantity: 150,
  price: 20000
}
{
  _id:
ObjectId('680dc2ece6dd67696266cd1d'),
  item: 'Office Chair',
  category: 'Furniture',
  quantity: 80,
  price: 7000
}
{
  _id:
ObjectId('680dc2ece6dd67696266cd1e'),
  item: 'Desk',
  category: 'Furniture',
  quantity: 50,
  price: 12000
}
{
  _id:
ObjectId('680dc2ece6dd67696266cd1f'),
  item: 'Water Bottle',
  category: 'Utilities',
  quantity: 500,
  price: 200
}
```

```

{
  _id:
  ObjectId('680dc2ece6dd67696266cd20'),
  item: 'Notebook',
  category: 'Stationery',
  quantity: 700,
  price: 40
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd21'),
  item: 'Pen',
  category: 'Stationery',
  quantity: 1000,
  price: 15
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd22'),
  item: 'Backpack',
  category: 'Bags',
  quantity: 90,
  price: 1500
}
{
  _id:
  ObjectId('680dc2ece6dd67696266cd23'),
  item: 'Printer',
  category: 'Electronics',
  quantity: 60,
  price: 18000
}

```

5. Limit to 5 documents:

```
db.stocks.find().limit(5)
```

Output:

```

{
  _id:
  ObjectId('680dc2ece6dd67696266cd1a'),
  item: 'Laptop',
  category: 'Electronics',
  quantity: 120,

```

```
    price: 55000
  }
  {
    _id:
    ObjectId('680dc2ece6dd67696266cd1b'),
    item: 'Smartphone',
    category: 'Electronics',
    quantity: 300,
    price: 25000
  }
  {
    _id:
    ObjectId('680dc2ece6dd67696266cd1c'),
    item: 'Tablet',
    category: 'Electronics',
    quantity: 150,
    price: 20000
  }
  {
    _id:
    ObjectId('680dc2ece6dd67696266cd1d'),
    item: 'Office Chair',
    category: 'Furniture',
    quantity: 80,
    price: 7000
  }
  {
    _id:
    ObjectId('680dc2ece6dd67696266cd1e'),
    item: 'Desk',
    category: 'Furniture',
    quantity: 50,
    price: 12000
  }
}
```

Result:

Thus, the above queries to perform Data Transformation on the 'Stock' collection in MongoDB have been executed successfully.

Ex. No. 9	ESTABLISHING CONNECTION BETWEEN MONGODB (COMPASS) AND JAVA(ECLIPSE)
Date:	

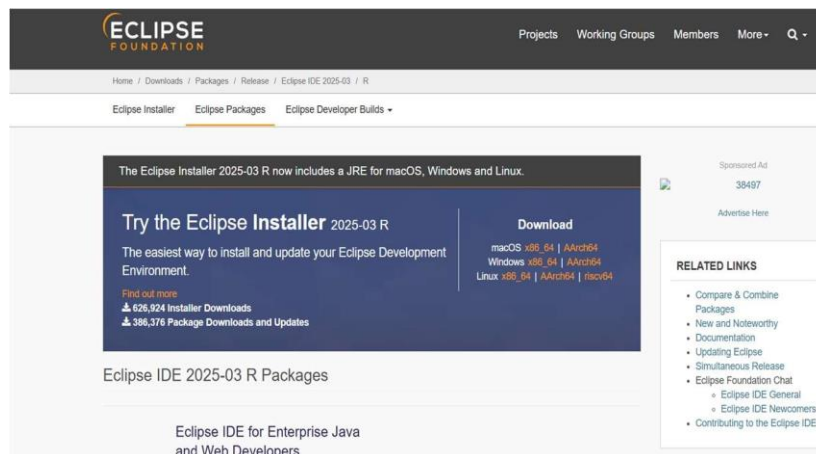
AIM:

To establish a connection between a Java program and a MongoDB database using Maven dependencies, insert a document into the database, and verify the insertion through MongoDB Compass.

PROCEDURE:

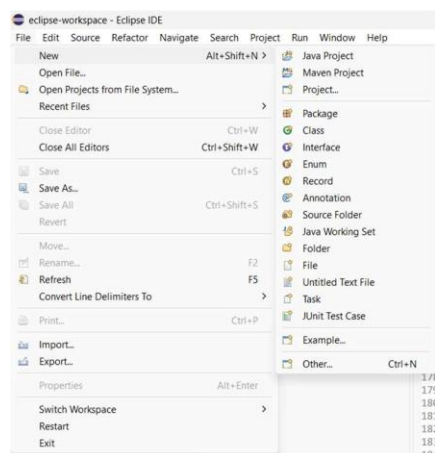
Step 1: Set Up the Environment

- Install **Eclipse IDE** from <https://eclipseide.org/> (if not already installed).



Step 2: Create a New Java Project

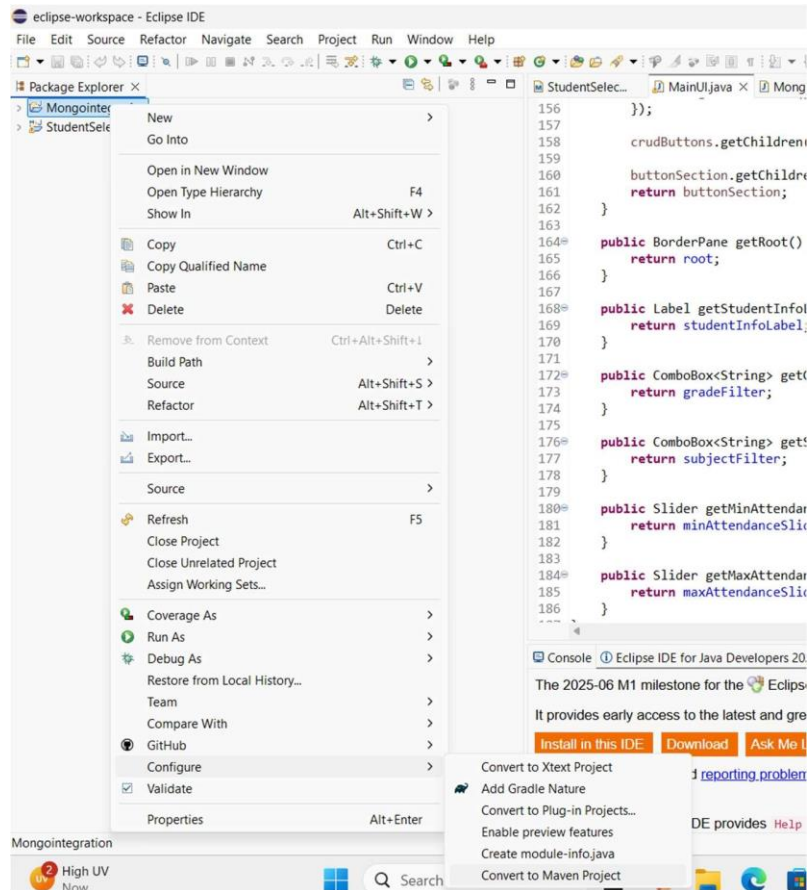
- Open Eclipse.
- Go to File > New > Java Project.



- Name the project (e.g., MongoIntegration).

Step 3: Convert to Maven Project

- Right-click the project > Configure > Convert to Maven Project.



Step 4: Add Maven Dependency

- Open the pom.xml file.
- Add the following dependency inside the <dependencies> tag:

```
<dependency>
  <groupId>org.mongodb</groupId>
  <artifactId>mongodb-driver-sync</artifactId>
  <version>4.11.0</version>
</dependency>
```

```

1 https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://m
3 <groupId>MongoIntegration</groupId>
4 <artifactId>MongoIntegration</artifactId>
5 <version>0.0.1-SNAPSHOT</version>
6 <build>
7 <sourceDirectory>src</sourceDirectory>
8 <plugins>
9 <plugin>
10 <artifactId>maven-compiler-plugin</artifactId>
11 <version>3.13.0</version>
12 <configuration>
13 <release>21</release>
14 </configuration>
15 </plugin>
16 </plugins>
17 </build>
18 </project>

```

- Save the pom.xml. Maven will automatically download the MongoDB driver library.

Step 5: Write Java Code

- Create a Java class (e.g., MongoConnect.java).
- Write the following code to connect to MongoDB, insert data, and retrieve documents:

```

import
com.mongodb.client.*;
import org.bson.Document;

public class MongoConnect {
    public static void main(String[] args) {
        String uri =
            "mongodb://localhost:27017";
        MongoClient mongoClient = MongoClient.create(uri);

        MongoDBDatabase database = mongoClient.getDatabase("testDB");
        MongoCollection<Document> collection = database.getCollection("students");

        Document doc = new Document("name", "Sahana")
            .append("age", 21)
            .append("department", "IT");
        collection.insertOne(doc);

        FindIterable<Document> docs =
            collection.find(); for (Document d : docs) {
            System.out.println(d.toJson());
        }

        mongoClient.close();
    }
}

```

Step 6: Run the Program

- Right-click the file > Run As > Java Application.
- Observe the console output displaying inserted documents.

Step 7: Verify in MongoDB Compass

- Open MongoDB Compass.
- Connect to mongodb://localhost:27017.
- Navigate to **testDB** > **students** collection.
- Verify that the document (e.g., Sahana, 21, IT) is inserted.

Result:

Thus, the connection between MongoDB (compass) and JAVA(eclipse) is established successfully.

Ex. No. 10	Develop a CRUD Application using MongoDB(Mini Project)
Date:	

Aim:

To develop a **Developer Knowledge Vault** web application using MongoDB as the database to store, manage, and retrieve structured coding knowledge such as concepts, bug fixes, and reusable code snippets.

Procedure:

Step 1 – Design and develop the frontend using React.js to provide an interactive user interface for managing knowledge notes.

Step 2 – Develop the backend using Java Spring Boot framework to handle business logic and RESTful API services.

Step 3 – Establish connection between Spring Boot and MongoDB using Spring Data MongoDB.

Step 4 – Design MongoDB collections for storing user data, knowledge notes, and version history.

Step 5 – Implement backend APIs for CRUD operations (Create, Read, Update, Delete) on notes and user profiles.

Step 6 – Integrate authentication using JWT to ensure secure access and user-specific data management.

Step 7 – Connect frontend with backend APIs to enable real-time interaction and data visualization.

Code:

Frontend (React – Example API Call):

```
useEffect(() => {
  axios.get("http://localhost:8080/api/memories")
    .then((res) => setMemories(res.data))
    .catch((err) => console.log(err));
}, []);
```

Backend (Spring Boot – Controller Example):

```
@GetMapping("/memories")
public List<Memory> getAllMemories() {
  return memoryService.getAllMemories();
}
```

MongoDB Model (Memory Collection):

@Document(collection = "memory")

public class Memory {

 @Id

 private String id;

 private String userId;

 private String title;

 private String content;

 private List<String> tags;

 private String category;

 private String difficulty;

 private Date createdAt;

 private Date lastAccessed;

 private int accessCount;

}

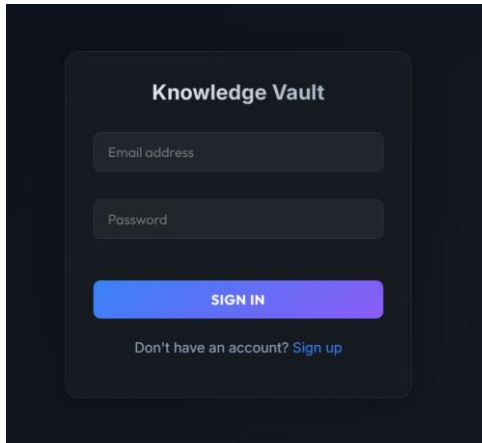
Repository Layer:

public interface MemoryRepository extends MongoRepository<Memory, String> {

 List<Memory> findByUserId(String userId);

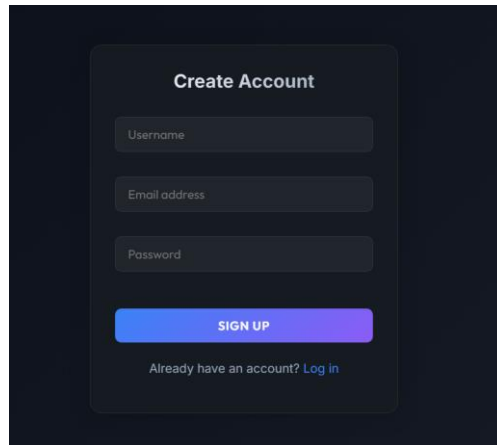
}

Output:



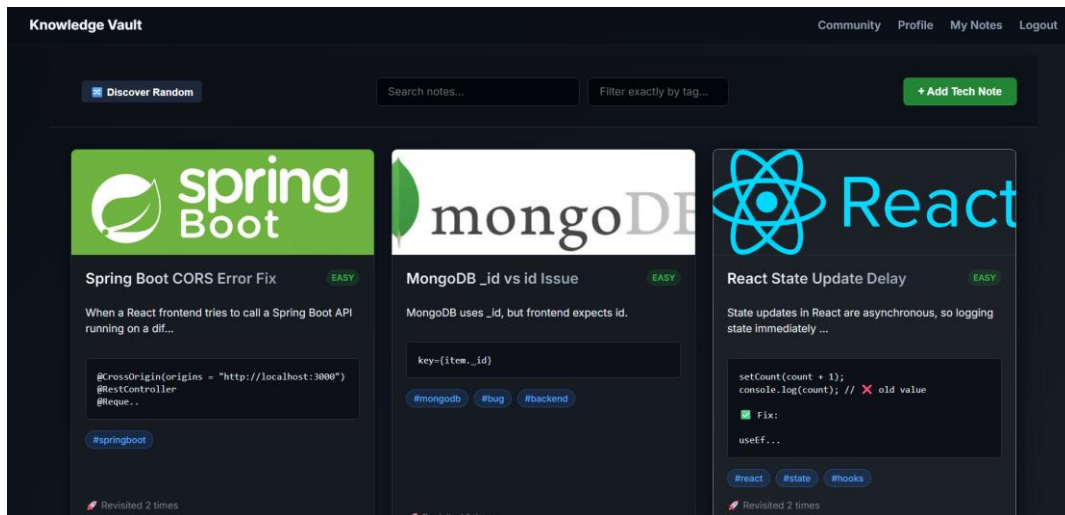
The login page features a dark blue background with a central white rounded rectangle. Inside, the title "Knowledge Vault" is at the top. Below it are two input fields: "Email address" and "Password". A large blue button with the text "SIGN IN" is positioned below the fields. At the bottom, a link says "Don't have an account? Sign up".

Login page

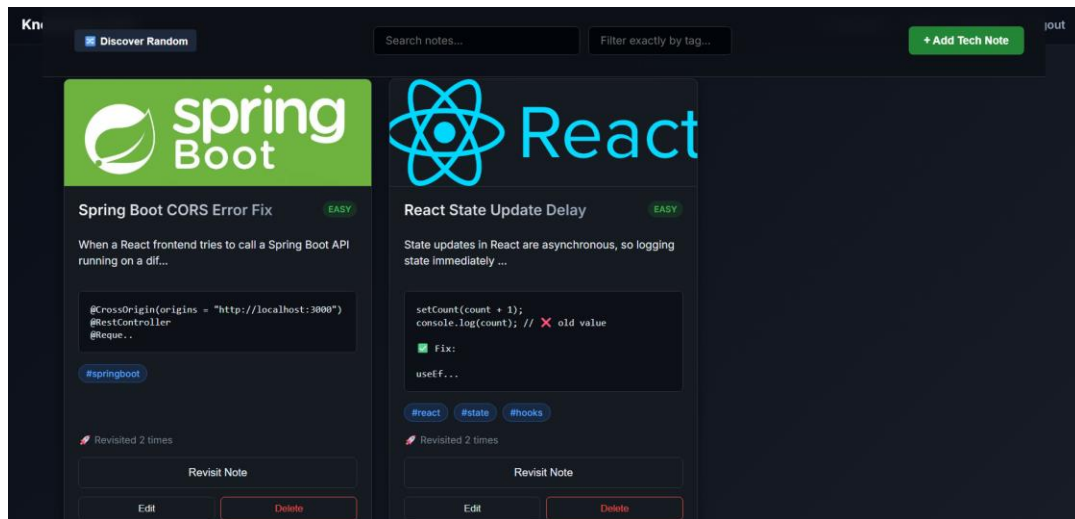


The register page has a dark blue background with a central white rounded rectangle. The title "Create Account" is at the top. Below it are three input fields: "Username", "Email address", and "Password". A large blue button with the text "SIGN UP" is positioned below the fields. At the bottom, a link says "Already have an account? Log in".

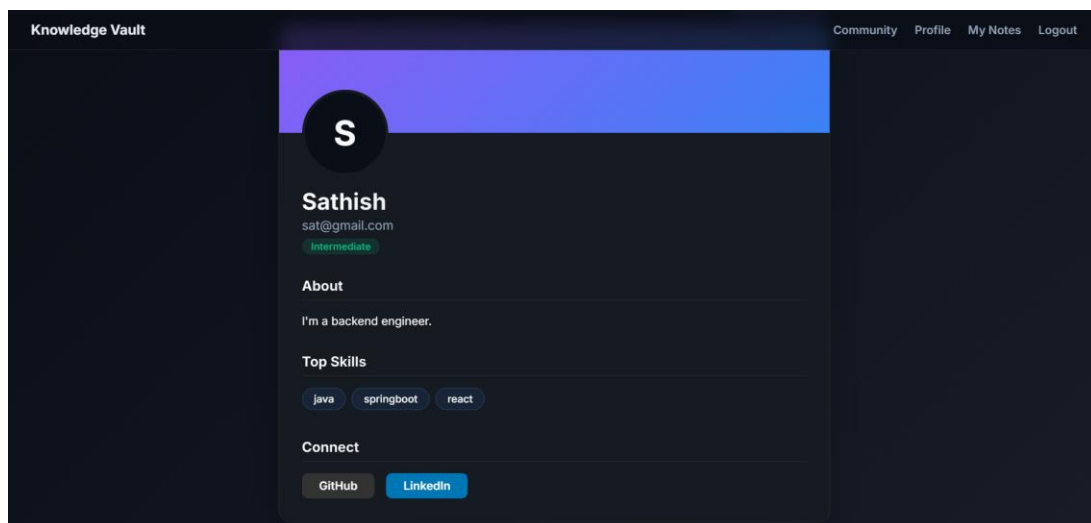
Register page



Community page



My notes page



Profile page

Result:

Thus, the Blog application with MongoDB as database is developed and verified successfully with React js as frontend and Node js as backend.